

Text Classification

Teaching a machine to put a label on a piece of text

Part A The classification pipeline, features, models & evaluation

Part B Naïve Bayes — your first classification algorithm

Supervised ML

BoW / TF-IDF

Naïve Bayes

"I love this movie."

 model →

Positive

"You won \$10,000 — claim now!"

 model →

Spam

"Pakistan won the final."

 model →

Sports

What we will cover today

A Text Classification

- What text classification is
- Where it is used in the real world
- The end-to-end pipeline
- Text → numbers (BoW, TF-IDF)
- Choosing a model, train/test split
- Binary vs multiclass vs multi-label
- Evaluating a classifier

B Naïve Bayes

- Bayes' theorem, refreshed
- Prior, likelihood, posterior
- Why the algorithm is "naïve"
- The zero-frequency problem
- Laplace smoothing
- A full worked calculation
- Multinomial NB in scikit-learn

By the end of this lecture you should be able to...



Define the task

Explain what text classification is and why it is a supervised learning problem.



Draw the pipeline

Go from raw text to a predicted label, naming every intermediate step.



Vectorise text

Convert sentences into numeric features using Bag of Words and TF-IDF.



Pick a model

Compare Naïve Bayes, Logistic Regression and SVM for text data.



Evaluate honestly

Use a confusion matrix, precision, recall and F1 — not accuracy alone.



Build one

Train a working sentiment classifier in a few lines of scikit-learn.

DEFINITION

What is text classification?

Text classification is the task of automatically assigning a **predefined category (label)** to a piece of text — a word, a sentence, a review, an e-mail or a whole document.

Text	Class
I love this movie.	Positive
This movie is amazing.	Positive
I hate this movie.	Negative
The movie was terrible.	Negative

The model learns the relationship $\text{text} \rightarrow \text{category}$ from these labelled examples.

AT TEST TIME

A new, unseen sentence arrives:

"The movie was fantastic."

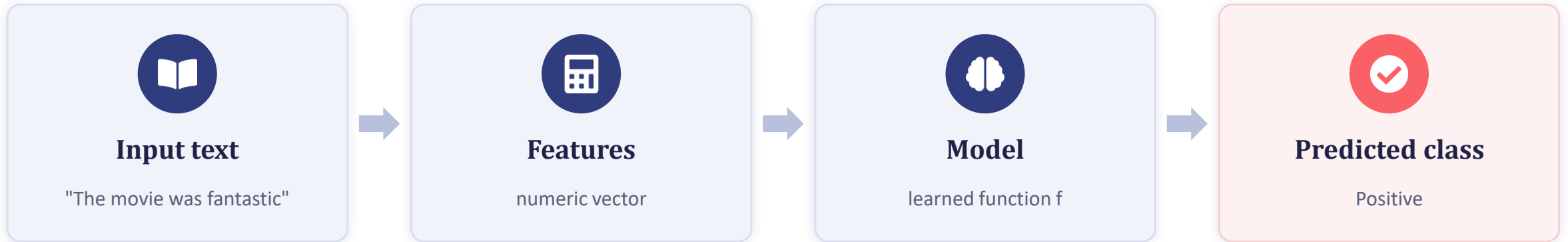


The trained model predicts

Positive

...even though this exact sentence was never seen during training.
That ability is called generalisation.

Text classification is supervised learning



What "supervised" means here

- We are given training pairs (x, y) : a document x and its correct label y .
- The algorithm searches for a function $f(x) \approx y$ that fits those examples.
- Success is measured on data the model has never seen — not on the training set.

Labels are the "supervision"

No labels $\rightarrow$ you cannot do classification.
You would be doing clustering instead (see later).

Where text classification shows up



Sentiment analysis

"The product is excellent." → Positive

Positive / Negative / Neutral



Spam detection

"Congratulations! You won \$10,000." → Spam

Spam / Not spam



News categorisation

"Pakistan defeated India in the final." → Sports

Sports / Politics / Business ...



Topic labelling

"The new processor has 16 cores." → Technology

What is this document about?



Intent detection

"Where is my order?" → Track_order

Powers chatbots & help desks



Content moderation

Flagging abusive or unsafe messages

Toxic / Safe

The complete text classification pipeline



Remember: a machine-learning algorithm cannot read words. Everything between the raw text and the model exists for one reason — to turn language into numbers without throwing away the information that decides the label.

STEP 1

Collect and label a dataset

ID	Review (input x)	Label (output y)
D1	I love this movie	Positive
D2	This movie is excellent	Positive
D3	I hate this movie	Negative
D4	This movie is terrible	Negative
D5	The movie is amazing	Positive
D6	The movie is boring	Negative

Input = the review · Output = the sentiment label

Where do labels come from?

Manual annotation · existing user signals (star ratings, "report spam" clicks) · public datasets such as IMDb reviews, 20 Newsgroups or AG News.

1 Balance the classes

Roughly equal examples per class, or the model just predicts the majority.

2 Label consistently

Two annotators must agree on what "Neutral" means. Write a labelling guideline.

3 Enough data

A few hundred examples per class is a realistic classroom minimum.

4 Keep it representative

Training data should look like the data you will meet in production.

Preprocessing: clean the text first

Raw text

"I REALLY love this movie!!!"

**Lowercase**

"i really love this movie!!!"

**Remove punctuation**

"i really love this movie"

**Tokenise**

["i", "really", "love", "this", "movie"]

**Remove stopwords**

["really", "love", "movie"]

Other common steps

- Stemming: studies → studi
- Lemmatisation: better → good
- Strip URLs, HTML, emojis, numbers
- Handle negation: not good → not_good



Do not preprocess blindly

Standard stopwords lists delete "not", "no" and "never".
For sentiment that is fatal:

"not good" and "good" collapse to the same features.
Always choose preprocessing to fit the task.

Turning text into numbers: Bag of Words

Vocabulary (from the whole corpus)

[i, love, hate, this, movie]

"I love this movie" → [1, 1, 0, 1, 1]

"I hate this movie" → [1, 0, 1, 1, 1]

"I love love this" → [1, 2, 0, 1, 0]

How it works

- Build one vocabulary of all unique words in the corpus.
- Each document becomes a vector of length $|V|$.
- Each position holds the count of that word in the document.
- Every document now has the same fixed length — exactly what ML models need.

Strengths

- Very simple
- Fast to compute
- Surprisingly strong baseline

Limitations

- Word order is lost
- Huge, sparse vectors
- No sense of meaning or similarity

TF-IDF: not all words are equally useful

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log(N / \text{DF}(t))$$

TF = how often term t appears in document d · DF = in how many documents t appears · N = total documents

1 Term Frequency (TF)

A word that occurs often in this document is probably important to it.

2 Inverse Document Frequency (IDF)

A word that occurs in every document ("the", "movie") carries little information — down-weight it.

3 The product

High score = frequent here, rare elsewhere. That is exactly a good discriminating feature.

Term	Appears in	IDF	Verdict
the	10 000 / 10 000 docs	0.00	useless
movie	9 000 / 10 000 docs	0.05	weak
excellent	600 / 10 000 docs	1.22	informative
dreadful	90 / 10 000 docs	2.05	very informative

log base 10, illustrative corpus of 10 000 reviews

In practice: start with TF-IDF. It is usually a clear win over raw counts for classification, and it is a one-line change in scikit-learn (TfidfVectorizer).

Four ways to represent text as numbers

Representation	Idea	Strength	Weakness
One-hot encoding	One 1 per word position in $ V $	Trivial to build	Enormous, no document-level view
Bag of Words	Word counts per document	Simple, strong baseline	Ignores order; common words dominate
TF-IDF	Counts weighted by rarity	Highlights discriminating words	Still ignores order and meaning
Word embeddings	Dense vectors learned from context	Captures similarity: king $\approx$ queen	Needs more data / compute



For this course

Focus on Bag of Words and TF-IDF — you already built both in Lectures 05–06.



Where the field went next

Word2Vec / GloVe $\rightarrow$ contextual embeddings (BERT). Same pipeline, richer features.

Choosing a classification algorithm



Naïve Bayes

Probabilistic. Counts how typical each word is of each class.

Fast, works on tiny datasets. Our next topic.



Logistic Regression

Learns a weight per feature and squashes the score into a probability.

The strong, sensible default for text.



Support Vector Machine

Finds the widest margin separating the classes.

Excellent on high-dimensional sparse text.



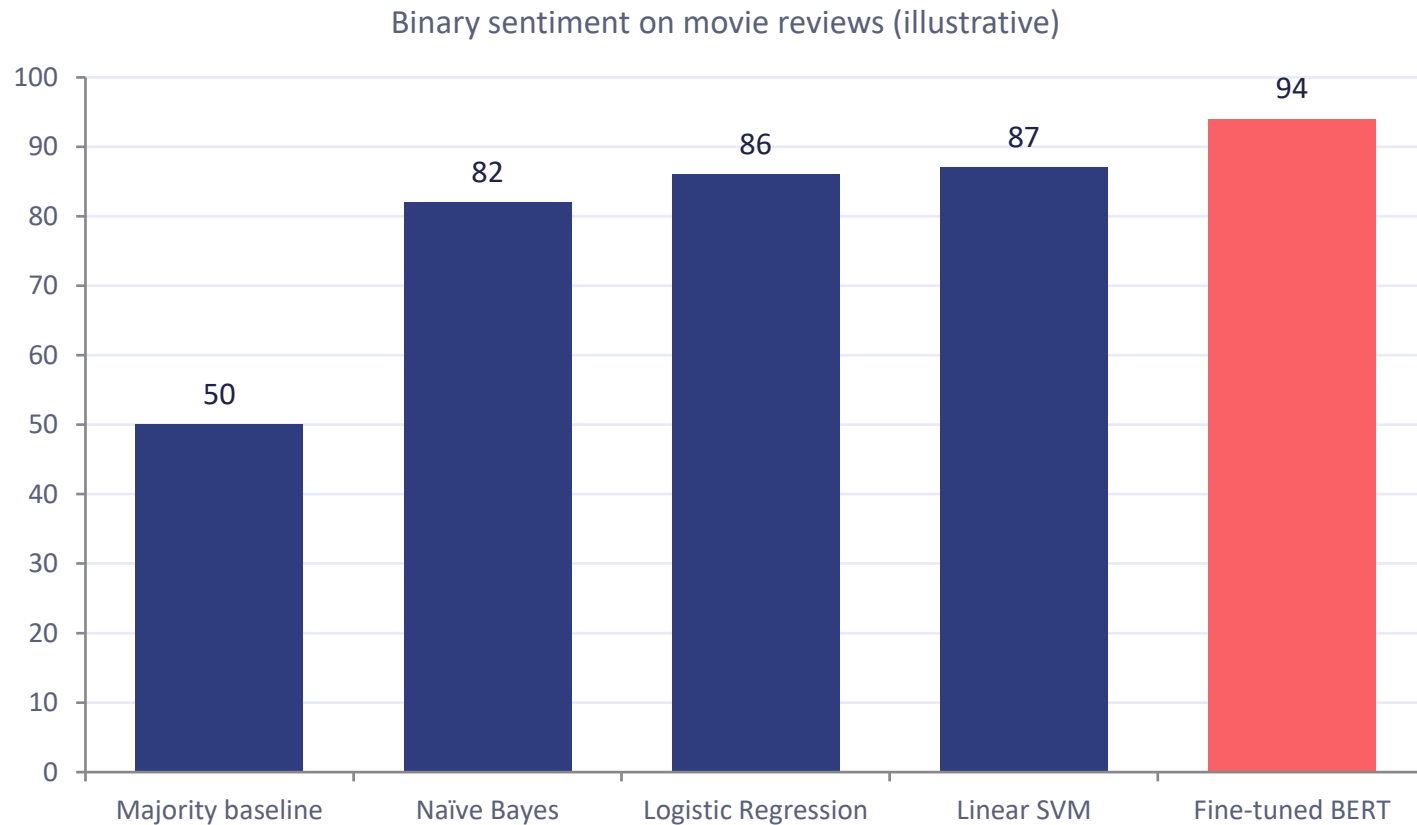
Neural Networks

Learns its own features through hidden layers.

Best with lots of data; CNN/RNN/Transformer.

Suggested learning order: Naïve Bayes → Logistic Regression → SVM → Neural networks. Always establish a simple baseline before reaching for a complex model.

What accuracy can you expect?



Always compare against a baseline

If 90% of e-mails are not spam, a model that predicts "not spam" every time already scores 90%.

Simple models are competitive

On small, clean, topic-driven datasets Naïve Bayes is within a few points of far heavier models.

Numbers depend on the dataset

These figures are typical ranges for a well-known benchmark, not guarantees for your data.

Never test on the data you trained on

1 000 labelled reviews

Training set — 80% (800 reviews)

Test 20%

Training data

The model sees the text and the labels, and learns from them.

Test data

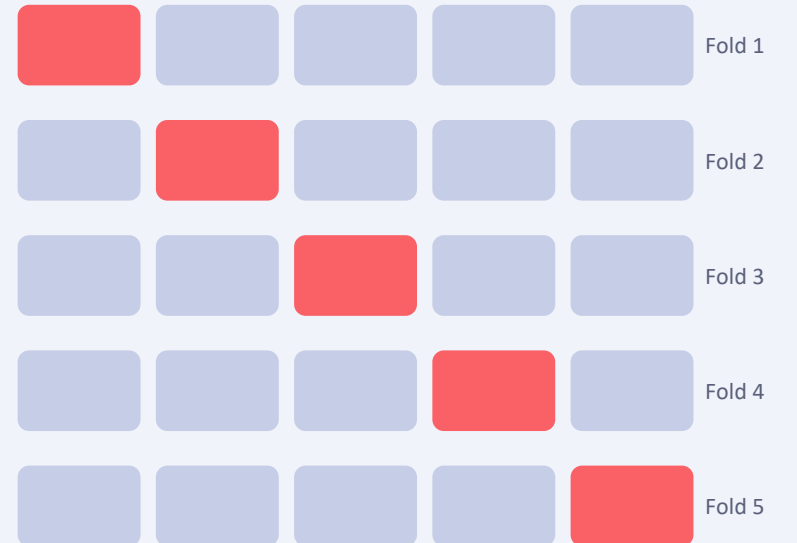
Held back completely. Used once, to estimate how the model generalises.



Testing on training data measures memorisation, not learning. The score will look wonderful and mean nothing.

k-fold cross-validation

With a small dataset, one split is luck. Split the data into k parts, train k times, and average the scores.



Red = held out for testing in that round

Binary vs multiclass vs multi-label

Binary

2 classes · pick 1

- Spam / Not spam
- Positive / Negative
- Toxic / Safe

"You won \$10,000" → Spam

Multiclass

> 2 classes · pick 1

- Sports
- Politics
- Business · Technology · Entertainment

"Pakistan won the final" → Sports

Multi-label

many classes · pick any number

- A document may carry several labels at once

"Pakistan's tech sector is growing fast" → Technology + Business

Multiclass: one document → exactly one class. **Multi-label:** one document → any number of classes (train one binary classifier per label).

Classification vs clustering



Classification

Supervised learning

- We provide the labels in advance.
- The model learns text → known label.
- Evaluation is objective: compare with the true label.

"I love it" → Positive



Clustering

Unsupervised learning

- No labels are given at all.
- The algorithm discovers groups by similarity.
- A human must inspect and name each group afterwards.

Group 1 · Group 2 · Group 3 → ? Sports, Politics, Tech

The confusion matrix

500 e-mails · 100 are truly spam

	Predicted Spam	Predicted Not spam
Actually spam	90 TP · True positive	10 FN · False negative — spam that got through
Actually not spam	20 FP · False positive — real mail sent to junk	380 TN · True negative

Read it as: rows = the truth, columns = what the model said. The diagonal is what it got right.

Why accuracy alone lies

Accuracy = $(90 + 380) / 500 = 94\%$. But a model that simply labels every e-mail "not spam" scores 80% on this data while catching nothing.

The two errors are not equally bad

- A false negative = one spam message in your inbox. Annoying.
- A false positive = your job offer lands in the junk folder. Serious.
- For medical screening the priority flips: missing a real case is the costly error.

Always decide which mistake you can least afford before you pick a metric.

Precision, recall and F1

Accuracy

$$(TP + TN) / \text{all}$$

94%

on our spam example

Fraction of all predictions that were right. Misleading when classes are imbalanced.

Precision

$$TP / (TP + FP)$$

0.82

on our spam example

Of everything flagged as spam, how much really was spam? Optimise when false alarms hurt.

Recall

$$TP / (TP + FN)$$

0.90

on our spam example

Of all the real spam, how much did we catch? Optimise when misses hurt.

F1-score

$$2PR / (P + R)$$

0.86

on our spam example

Harmonic mean of precision and recall — one number when you need a balance.

Rule of thumb: report accuracy only alongside the confusion matrix; report macro-averaged F1 when the classes are imbalanced or when every class matters equally.

Four mistakes beginners make



Class imbalance

95% of your data is one class, so the model learns to always guess it.
Fix: resample, class weights, or judge by F1 instead of accuracy.



Data leakage

Fitting the vectoriser on the full dataset before splitting lets test-set vocabulary leak into training. Fit on train only.



Overfitting

Near-perfect training accuracy with a poor test score. The model memorised examples instead of learning patterns. Simplify or add data.



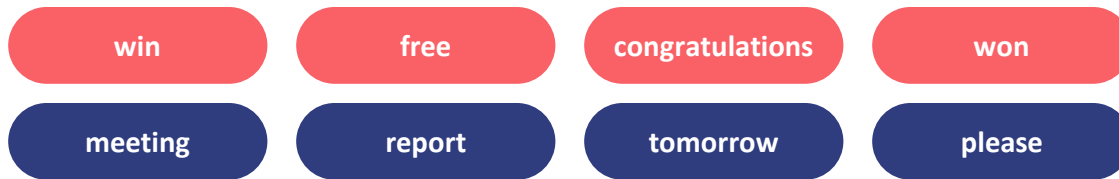
Ignoring the errors

The single most useful debugging step is reading the documents your model got wrong. The pattern is usually obvious.

A small spam classifier, end to end

Training message	Label
Win a free iPhone now	Spam
Congratulations you won	Spam
Meeting at 10 AM tomorrow	Not spam
Please send the report	Not spam

The model learns which words carry evidence for each class:



coral = evidence for Spam · navy = evidence for Not spam

NEW, UNSEEN MESSAGE

"Congratulations! You won a free phone."

The classifier outputs probabilities



Prediction = Spam

The class with the highest probability wins. How those probabilities are computed is exactly what Part B answers.

PART B

Naïve Bayes for NLP

We know the pipeline. Now: how does the model actually decide the class?

Bayes' theorem → prior + likelihood → posterior → highest probability wins



Bayes' theorem

$$P(C | X) = \frac{P(X | C) \times P(C)}{P(X)}$$

$P(C | X)$

Posterior

How likely is class C, now that I have seen this text?

$P(X | C)$

Likelihood

How typical is this text for documents of class C?

$P(C)$

Prior

How common is class C before looking at any words?

$P(X)$

Evidence

Probability of the text itself — the same for every class, so we can ignore it.

For classification we compare $P(C | X)$ across the classes. Because $P(X)$ is identical for all of them, we only need to rank **$P(C) \times P(X | C)$** and pick the winner.

Why is it called "naïve"?

The independence assumption

Given the class, every word is assumed to appear independently of every other word — and its position in the sentence is ignored.

Obviously false in real language

- "New York" — the two words are anything but independent.
- "not good" and "good not" get identical features.
- The probability estimates it produces are often badly over-confident.
- And yet the ranking of classes is usually still correct — which is all we need.

$$P(C \mid \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n) \propto P(C) \times \prod P(\mathbf{x}_i \mid C)$$

For the document "excellent movie":

$$P(\text{Pos}) \times P(\text{excellent} \mid \text{Pos}) \times P(\text{movie} \mid \text{Pos})$$

$$P(\text{Neg}) \times P(\text{excellent} \mid \text{Neg}) \times P(\text{movie} \mid \text{Neg})$$



Why it still works

We do not need the true probability — only the correct ordering of the classes. The assumption also keeps the model tiny and trainable from a handful of examples.

The zero-frequency problem and Laplace smoothing



The problem

Suppose the word "excellent" never appears in any Negative training document. Then $P(\text{excellent} \mid \text{Negative}) = 0$, and because Naïve Bayes multiplies:

$$P(\text{Neg}) \times 0 \times P(\text{movie} \mid \text{Neg}) \times \dots = 0$$

One unseen word wipes out the entire class — no matter how much evidence the other words provided.

Also note: at prediction time, words never seen in training at all are simply dropped from the calculation.

The fix: Laplace (add-one) smoothing

$$P(w \mid C) = \frac{\text{count}(w, C) + 1}{\text{count}(C) + |V|}$$

- $\text{count}(w, C)$ — times word w occurs in class C
- $\text{count}(C)$ — total word occurrences in class C
- $|V|$ — vocabulary size, added to keep the probabilities summing to 1
- $+1$ — pretend every word was seen once more than it was

No probability can ever be zero again. Adding a constant α instead of 1 gives Lidstone smoothing — in scikit-learn that is the alpha parameter.

Counting: priors, vocabulary and word counts

ID	Text	Class
D1	good movie	Positive
D2	good film	Positive
D3	excellent movie	Positive
D4	bad movie	Negative
D5	bad film	Negative
D6	terrible movie	Negative

Vocabulary $V = \{ \text{good, movie, film, excellent, bad, terrible} \}$
 $|V| = 6$

Priors: $P(\text{Positive}) = 3/6 = 0.5$ $P(\text{Negative}) = 3/6 = 0.5$

Word	Count in Positive	Count in Negative
good	2	0
movie	2	2
film	1	1
excellent	1	0
bad	0	2
terrible	0	1
Total words	6	6

Notice the zeros. "good" never occurs in a Negative document. Without smoothing the Negative posterior would collapse to exactly 0 — so we apply Laplace smoothing on the next slide.

Classifying the new document "good movie"

Smoothed likelihood: $P(w \mid C) = (\text{count}(w, C) + 1) / (6 + 6) = (\text{count} + 1) / 12$

Positive

$$P(\text{Pos}) = 0.5$$

$$P(\text{good} \mid \text{Pos}) = (2+1)/12 = 0.250$$

$$P(\text{movie} \mid \text{Pos}) = (2+1)/12 = 0.250$$

$$0.5 \times 0.250 \times 0.250 = 0.03125$$

Negative

$$P(\text{Neg}) = 0.5$$

$$P(\text{good} \mid \text{Neg}) = (0+1)/12 = 0.083$$

$$P(\text{movie} \mid \text{Neg}) = (2+1)/12 = 0.250$$

$$0.5 \times 0.083 \times 0.250 = 0.01042$$

Normalise: $0.03125 / (0.03125 + 0.01042) = 0.75 \rightarrow P(\text{Positive} \mid \text{"good movie"}) = 75\%$

Prediction = Positive

Multinomial Naïve Bayes and its siblings

Multinomial NB

Word counts / TF-IDF weights

The standard choice for text classification.

Bernoulli NB

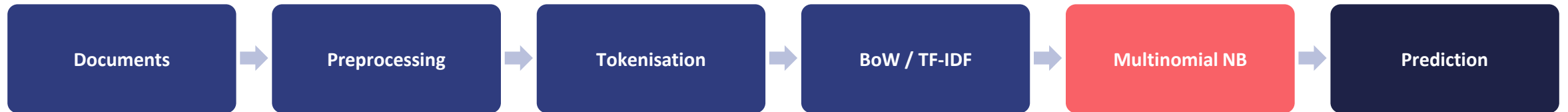
Binary word present / absent

Useful for very short texts such as tweets or SMS.

Gaussian NB

Continuous numeric features

For real-valued features, not raw word counts.



Extremely fast

Training is just counting — it scales to millions of documents.

Works on small data

A few dozen examples per class already give something usable.

A strong baseline

Beat it before you claim your fancy model is worth the complexity.

The whole thing in scikit-learn

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

texts = ["I love this movie",
         "This movie is excellent",
         "I hate this movie",
         "This movie is terrible"]

labels = ["Positive", "Positive", "Negative", "Negative"]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts)

model = MultinomialNB()
model.fit(X, labels)

X_new = vectorizer.transform(["This movie is excellent"])
print(model.predict(X_new))      # ['Positive']
```

Two lines to upgrade it

- TfidfVectorizer() instead of CountVectorizer()
- ngram_range=(1,2) to keep bigrams like "not good"
- MultinomialNB(alpha=1.0) — that alpha is Laplace smoothing

Then evaluate properly

```
train_test_split(...)
classification_report(y_test, y_pred)
confusion_matrix(y_test, y_pred)
```

Wrap it all in a Pipeline so the vectoriser is fitted on the training fold only.

Classroom activity (15–20 minutes)

Dataset

D1	this product is excellent	Positive
D2	I love this product	Positive
D3	this product is amazing	Positive
D4	this product is terrible	Negative
D5	I hate this product	Negative
D6	this product is useless	Negative

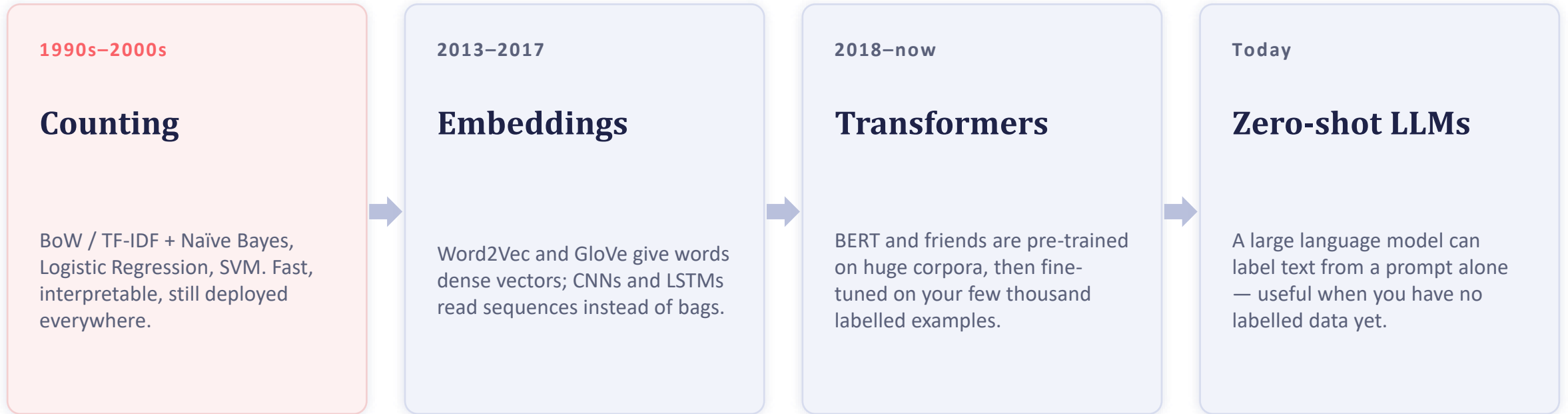
Test document: "I love this excellent product"

Then check your answer in three lines of scikit-learn — hand calculation first, code second.

Do this by hand

- 1 Tokenise all six documents
- 2 Build the vocabulary and give $|V|$
- 3 Write the Bag of Words vector for D1 and D4
- 4 Compute the two priors
- 5 Compute the Laplace-smoothed likelihoods
- 6 Compute both posteriors and state the prediction

Where this sits in modern NLP



The pipeline never changed — only the box labelled "feature extraction" did. Understanding it on Bag of Words is what lets you understand it on BERT.

Five things to remember

- 1 Text classification = assigning a predefined label to text. It is supervised learning, so it needs labelled data.
- 2 The pipeline is always the same: clean → tokenise → vectorise → train → evaluate → predict.
- 3 BoW and TF-IDF are how words become numbers; TF-IDF rewards words that are frequent here and rare elsewhere.
- 4 Naïve Bayes picks the class with the highest $P(\text{class}) \times \prod P(\text{word} \mid \text{class})$, and Laplace smoothing keeps a single unseen word from zeroing everything out.
- 5 Never test on training data, and never trust accuracy alone — read the confusion matrix.

Questions?

Next up: build the sentiment classifier yourself — TF-IDF + Logistic Regression on a real review dataset, then compare it against Naïve Bayes.

Read: SLP3 Ch. 4

Practice: sklearn tutorial

